AFRILANG Stdlib

std/m/queryparse

Bibliothèque AFRILANG `std/m/queryparse` (niveau medium, catégorie medium). Elle expose 4 fonction(s) : parseQuery, getP

`import "std/m/queryparse"` · `use queryparse`

- `parseQuery(query text) ? list text`
- `getParam(query text, key text) ? text`
- `paramCount(query text) ? number`
- `hasParam(query text, key text) ? bool`

Category: medium | Tier: medium | Functions: 4

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/m/queryparse` (niveau medium, catégorie medium). Elle expose 4 fonction(s) : parseQuery, getP

`import "std/m/queryparse"` · `use queryparse`

- `parseQuery(query text) ? list text`
- `getParam(query text, key text) ? text`
- `paramCount(query text) ? number`
- `hasParam(query text, key text) ? bool`

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/m/queryparse"
use queryparse
```

Niveau : medium · Catégorie : Modules medium (m/)
Bibliothèques intermédiaires ? import std/m/?

3. Référence API

? parseQuery(query text) ? list
? getParam(query text, key text) ? text
? paramCount(query text) ? number
? hasParam(query text, key text) ? bool

4. Exemples d'utilisation

```
import "std/m/queryparse"
use queryparse

create result = parseQuery(/* args */)
say result

create result = getParam(/* args */)
say result

create result = paramCount(/* args */)
say result

create result = hasParam(/* args */)
say result
```

5. Bonnes pratiques

? Préférez `import "std/m/queryparse"` en tête de fichier.
? Lancez `afrilang check` avant de livrer.
? Combinez avec les modules stdlib de la même catégorie.
? Voir /stdlib/ pour le source live et les démos playground.
? Signalez les problèmes sur GitHub si une export manque.

6. Annexe ? source

```
module queryparse

function parseQuery(query text) returns list text
end

function getParam(query text, key text) returns text
end

function paramCount(query text) returns number
end

function hasParam(query text, key text) returns bool
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/m/queryparse` (tier medium, category medium). Exports 4 function(s): parseQuery, getParam, paramCo

`import "std/m/queryparse"` · `use queryparse`

- `parseQuery(query text) ? list text`
- `getParam(query text, key text) ? text`
- `paramCount(query text) ? number`
- `hasParam(query text, key text) ? bool`

2. Installation & import

Add the module to your program:

```
import "std/m/queryparse"
use queryparse
```

Tier: medium · Category: Medium modules (m/)
Intermediate libraries ? import std/m/?

3. API reference

? parseQuery(query text) ? list
? getParam(query text, key text) ? text
? paramCount(query text) ? number
? hasParam(query text, key text) ? bool

4. Usage examples

```
import "std/m/queryparse"
use queryparse

create result = parseQuery(/* args */)
say result

create result = getParam(/* args */)
say result

create result = paramCount(/* args */)
say result

create result = hasParam(/* args */)
say result
```

5. Best practices

? Prefer `import "std/m/queryparse"` at the top of your file.
? Run `afrilang check` before shipping.
? Combine with related stdlib modules from the same category.
? See /stdlib/ for the live source and playground demos.
? Report issues on GitHub if an export is missing or undocumented.

6. Appendix ? source

```
module queryparse

function parseQuery(query text) returns list text
end

function getParam(query text, key text) returns text
end

function paramCount(query text) returns number
end

function hasParam(query text, key text) returns bool
end
end
```